

TALLER 1: ALGORITMOS PARALELOS
Curso: Infraestructura paralelas y distribuidas
Semestre: 2026-1 (febrero - junio)

Nombre: Isabela Bermúdez Moreno
código: 202418564-3743

Parte 1: Preguntas Conceptuales (50%)

1. Motivación para la Programación Paralela:

• **¿Por qué es importante la programación paralela en la informática moderna?**

R// La programación paralela permite dividir un problema computacional en tareas más pequeñas, que pueden ser procesadas simultáneamente en múltiples procesadores. Esto ayuda a minimizar el tiempo de ejecución de algoritmos. Por ende, se produce un ahorro de tiempo, al permitir ejecutar código más rápido, y dinero, ya que se aprovecha al máximo los recursos de los procesadores. Es importante en la informática moderna porque permite optimizar los sistemas y ayuda a resolver problemas complejos .

• **Da ejemplos de aplicaciones o áreas donde la programación paralela es esencial.**

R//

- **Inteligencia artificial:** En el machine learning es necesario procesar grandes cantidades de datos a la vez en GPU'S con miles de núcleos.
- **Motores de búsqueda como Google:** Deben buscar información en millones de páginas web al mismo tiempo para dar una respuesta lo más rápido posible.
- **Videojuegos:** Se usa el paralelismo para renderizar gráficos en tiempo real
- **Simulaciones:** Se usa la computación paralela para resolver cálculos masivos en modelos como la predicción del clima.

2. Conceptos Fundamentales:

• **Explica la analogía de los cocineros y los clientes para ilustrar la paralelización.**

R// La analogía consiste en imaginar un restaurante que cuenta con solo un cocinero, el cual se encarga de atender los clientes, hacer la comida y llevar los pedidos. Inevitablemente, el cocinero no dará abasto a todos los clientes que llegan, lo que genera retraso en los pedidos y en general, mala atención al cliente. Por el contrario, se tiene un restaurante que cuenta con más de un cocinero y varios meseros. El personal será capaz de atender a los clientes

de manera satisfactoria, serán más rápidos en anotar las órdenes, hacer la comida y llevar los pedidos.

Podemos comparar esta situación con la computación. Si un algoritmo es ejecutado por solo un procesador o núcleo, las tareas se van completando una por una, de manera secuencial, hasta llegar al final. En algoritmos con pequeños conjuntos de datos, no habrá problema, sin embargo, cuando se trabaja con grandes cantidades de datos, el tiempo de ejecución puede demorar mucho y los recursos del procesador no se aprovechan al máximo. Si en cambio, dividimos un algoritmo en pequeñas subtareas, y las ejecutamos a la vez en varios núcleos, el tiempo de respuesta mejorará cuando trabajemos con esos grandes conjuntos de datos. La clave es dividir un problema mayor en subproblemas que pueden ejecutarse a la vez.

• **¿Cómo ha evolucionado la computación paralela desde sus inicios hasta la actualidad?**

R// El paralelismo, es un concepto que surgió incluso antes de que se pudiera aplicar. En 1958, cuatro investigadores de IBM, cocke y slotnick discutieron por primera vez el uso del paralelismo en cálculos numéricos.

En 1964, dos años después de la creación de ATLAS, la primera máquina que empleó la memoria virtual y paginación, la fuerza aérea estadounidense financió la creación del primer computador paralelo masivo ILLIAC IV, con 256 procesadores.

En 1967, amdahl y slotnick debatieron sobre la viabilidad del procesamiento en paralelo, dando paso a la ley de amdahl.

Desde 1968 hasta 1976, se hacen diferentes proyectos en investigación sobre el paralelismo. A mediados de los 80, un nuevo tipo de computador paralelo fue creado para aplicaciones científicas. A final de los años 80, aparecen los clúster.

A partir de los años 2000, el paralelismo se hizo más accesible con la llegada de los procesadores multinúcleo. Surgen nuevos tipos de paralelización, dependiendo de si una o más instrucciones se ejecutan a la vez, y si se procesa uno o múltiples datos a la vez (ej: SISD, SIMD, MISD, MIMD). Más adelante, el paralelismo se pudo aplicar también en los GPUs, usado en la actualidad en diversas áreas, que van desde la inteligencia artificial hasta los videojuegos. También surge la computación distribuida, donde varias máquinas trabajan en paralelo por medio de la red.

3. Patrones de Programación Paralela:

- **¿Por qué son importantes los patrones de programación paralela?**

R// Los patrones de programación paralela son un conjunto de buenas practicas a la hora de optimizar programas. Al conocer los patrones de paralelización, podemos discernir cual utilizar en los problemas que debamos resolver, lo que nos ahorra tiempo y esfuerzo a la hora de programar. Además los patrones ofrecen soluciones que han sido probadas y que sirven para resolver problemas complejos y aprovechar los recursos de hardware al máximo.

- **Menciona algunos beneficios de utilizar patrones y frameworks paralelos.**

R// Los beneficios de utilizar patrones y frameworks paralelos son:

- Reducción de tiempos de ejecución
- Mayor rendimiento
- Aprovechamiento al máximo de los recursos
- Ofrecen soluciones probadas
- Mayor escalabilidad
- Reducción de errores críticos
- Ahorro de tiempos de desarrollo
- Automatización de tareas en el caso de los frameworks paralelos

4. Ley de Amdahl:

- **Explica la Ley de Amdahl y cómo afecta el rendimiento al paralelizar tareas.**

R// Toda tarea tiene una parte secuencial y paralelizable. La ley de amdahl nos dice que la máxima aceleración que obtenemos de paralelizar una tarea siempre va a estar limitada por las partes secuenciales, por más procesadores que tengamos.

Disponer de múltiples procesadores no significa tener mayor rendimiento, debido a que llega un punto en donde por más procesadores que se tenga, la aceleración se va a comportar de manera lineal, limitada por la parte secuencial. Entonces, la ley de amdahl nos permite calcular en teoría, la máxima aceleración de un programa, lo que nos ayuda a saber cuando vale la pena paralelizar una tarea y cuantos procesadores realmente necesitamos para llegar a una buena aceleración.

La fórmula de la ley de Amdahl es:

$$S = 1 / (1 - P + P/N)$$

donde S es la aceleración, P es la parte paralelizable del código y N es el número de procesadores.

- **¿Qué implicaciones tiene esta ley en la optimización de programas paralelos?**

R// Esta ley nos permite saber hasta qué punto optimizar programas paralelos es rentable. Teniendo en cuenta que múltiples procesadores no significa mayor rendimiento, la ley de amdahl nos ayuda a identificar qué partes de un programa vale la pena optimizar, hasta qué punto la aceleración va a estar limitada por la parte secuencial y cuántos recursos de cómputo usar para paralelizar la tarea.

5. Trampas Seriales:

- **¿Qué son las "trampas seriales" en la programación paralela?**

R// Las trampas seriales son esas partes del código de un programa que deben ejecutarse estrictamente en secuencial. Es común que programadores caigan en estas trampas e intente paralelizar el código, lo que resulta en errores y pérdida de tiempo.

- **¿Por qué es importante identificarlas y evitarlas?**

R// Por que identificar las partes del código que se pueden o no paralelizar es fundamental a la hora de optimizar los programas. Reconocer las trampas seriales permiten al programador o arquitecto saber en donde debe invertir su tiempo, si vale la pena intentar paralelizar un algoritmo y discernir que patrón de diseño paralelo le puede resultar útil implementar.

6. Work-Span:

- **Define los conceptos de "Work" y "Span" en el contexto de algoritmos paralelos.**

R// Work es la métrica usada para representar el tiempo total requerido en ejecutar un algoritmo en un solo procesador, mientras que Span es el tiempo mínimo que se requiere para ejecutar un algoritmo en un numero ilimitado de procesadores.

- **¿Cómo se relaciona el Span con la ruta crítica de un programa paralelo?**

R// La ruta crítica de un programa es la serie de pasos o tareas más larga y que el procesador no puede ejecutar al mismo tiempo, es decir que dependen una de las otras. En general, un algoritmo se puede representar como un grafo Acíclico Dirigido (DAG), en donde los nodos son operaciones y las flechas las dependencias entre ellas, la ruta crítica entonces representa el camino más largo del grafo y el span mide el tiempo mínimo necesario para procesar esta ruta con un número ilimitado de procesadores. Debido a esto, el span nos ayuda a saber la aceleración máxima que podemos obtener al paralelizar un programa.

7. Memoria compartida, localidad y cache:

- **Explica cómo la memoria compartida y la localidad pueden mejorar los tiempos de ejecución en programas paralelos.**

R// La memoria compartida es un espacio de memoria en la cual varios procesadores pueden acceder a ella, generalmente por medio de un bus de memoria. Esto facilita la comunicación entre procesos, ya que permite que varios procesos puedan acceder y modificar directamente los datos compartidos de manera más rápida y sencilla. En la programación paralela, la comunicación entre distintos procesos o hilos es fundamental, y la memoria compartida es una herramienta para que se pueda dar.

Por otro lado, la localidad o localidad de referencia es un principio que nos dice que los programas tienden a acceder a los mismos datos o instrucciones si ya lo han hecho anteriormente y si ya se accedió a una ubicación de memoria específica, es probable que también se acceda pronto a las ubicaciones cercanas. Bajo este principio funciona la memoria caché, que guarda la información que es mas probable que se use a futuro.

Los programas paralelos con una fuerte localidad se ejecutan más rápido por que utilizan la memoria caché de forma más eficaz y se ahorran el tiempo que se gastarían accediendo a las otras memorias.

- **¿Qué papel juega la caché en la optimización del rendimiento?**

R// La memoria caché impacta directamente en el rendimiento de un programa. Si el programa tiene una fuerte localidad de referencia, la caché acelera el rendimiento al guardar copias de los datos e instrucciones de uso frecuente, lo que reduce la latencia de acceso, es decir evita que el CPU tenga que ir a buscar la información en la RAM cada vez que lo necesita.

Por el contrario si el programa a ejecutar accede a muchos datos aleatorios, es decir que tiene baja localidad, se puede ralentizar el rendimiento, ya que la memoria caché no estaría siendo útil.

Parte 2: Reto de Programación (50%) Objetivo Específico:

Repositorio en git-hub con el código: [Repositorio con código](#)

Descripción del Reto y respuestas:

1. Cálculo de Fibonacci: Implementa una función para calcular el n-ésimo número de Fibonacci.

Se implementó la función fibonacci(n) de forma recursiva. El caso base es cuando n es menor o igual a 1, en cuyo caso devuelve n (0 si n es 0 y 1 si n es 1). La parte recursiva se da cuando n es mayor que 1: en este caso la función devuelve la suma de los dos números de Fibonacci anteriores, con los llamados recursivos fibonacci(n-1) + fibonacci(n-2)

2. Paralelización de Ciclos: Utiliza `concurrent.futures` para paralelizar el cálculo de una serie de números de Fibonacci (por ejemplo, los primeros 20 números). Identifica cuáles ciclos `for` son paralelizables y cuáles no.

Se usó concurrent.futures para paralelizar el cálculo de la serie de números de Fibonacci. Los ciclos for que se pueden paralelizar son aquellos cuyas iteraciones no dependen del resultado de otras iteraciones. En este caso, el ciclo que recorre range(n_elementos + 1) y llama a executor.submit(fibonacci, i) es paralelizable, ya que cada fibonacci(i) solo depende de i. El hilo principal solo entrega las tareas (algo muy rápido) y el cálculo pesado se reparte entre los hilos o procesos del executor, que lo ejecutan al mismo tiempo.

Como caso contrario, el for que recorre la lista de futures en el orden en que fueron creados y llama a future.result() se dejó secuencial. Su función es recolectar los resultados en orden, y para eso future.result() tiene que esperar a que cada tarea termine. La impresión tampoco se paraleliza: se hace una sola vez, en el hilo principal, por que la salida debe ir en orden.

3. Evitar Trampas Seriales: Presta especial atención a la impresión de los resultados. La salida debe ser ordenada y correcta, incluso con la paralelización. Evita el uso directo de `print` dentro de los hilos/procesos para evitar conflictos.

La trampa serial sucede cuando se imprime o se guarden los resultados dentro de los hilos o procesos, ya que esto no garantiza que la salida sea ordenada. En el ejemplo de código base, los resultados no se imprimen en orden, ya que el arreglo iba guardando los números según el orden en que iban acabando los hilos/procesos.

Para evitarlo, los workers solo calculan y retornan el valor, sin imprimir nada. El hilo principal guarda los futures (objetos que representan un resultado pendiente) en una lista, en el orden en que fueron creados, y luego la recorre llamando `future.result()`, que espera si la tarea todavía no ha terminado. Así los resultados se guardan en el orden correcto y se imprimen en el hilo principal al final del programa.

4. Medición de Rendimiento: Mide y muestra el tiempo de ejecución total del programa.

A continuación se mostraran los resultados que se obtienen para los valores de $N=10,20,30,35$ y 40 al ejecutar el algoritmo en hilos y en procesos.

$n = 10$
hilos: 0.0012 segundos
procesos:

$n = 20$
hilos: 0.0035 segundos
procesos:

$n = 30$
hilos: 0.2119 segundos
procesos:

$n = 35$
hilos: 2.3857 segundos
procesos: 1.1835 segundos

$n = 40$
hilos: 26.2123 segundos
procesos: 11.4000 segundos

Se puede observar que para valores pequeños, los hilos son más rápidos que los procesos, por que es más costoso crear los procesos.

A partir de $n=35$, los procesos son más rápidos, ya que en python los hilos no pueden ejecutar en paralelo, hacen un trabajo concurrente, a diferencia de los procesos que pueden correr en distintos núcleos a la vez.

5. Análisis: Justifica qué ciclos `for` se pueden paralelizar y cuáles no, explicando el porqué.

Se puede paralelizar el ciclo for que llama a `executor.submit(fibonacci, i)` para cada `i`, ya que cada iteración usa su propio `i` y no depende del resultado de otro ciclo. Por eso las iteraciones pueden ejecutarse al mismo tiempo y en cualquier orden. No se puede paralelizar el ciclo que recorre a los futures y llama a `future.result()`, por que debe ser secuencial si se quiere guardar los resultados en orden. No se puede paralelizar un for para imprimir los resultados, ya que la salida debe ser en orden.

Preguntas Adicionales (para reflexionar después del reto):

1. ¿Cuáles ciclos `for` en el código consideraste paralelizables? ¿Por qué?

Consideré paralelizable el ciclo for que llama a `executor.submit(fibonacci, i)` para cada `i`, ya que el cálculo del Fibonacci de un número no depende del resultado de otro, por lo que se pueden ejecutar simultáneamente en diferentes hilos o procesos y en cualquier orden.

2. ¿Cómo evitaste las trampas seriales, especialmente en la impresión de resultados?

Los hilos/procesos solo calculan y retornan el valor, nunca imprimen. Usé el objeto `future` y su función `future.result()` para recolectar los resultados en el orden en que los futures fueron creados por el hilo principal, y hasta que están todos, imprimo la lista completa una sola vez.

3. ¿Qué impacto tuvo la paralelización en el rendimiento del cálculo de Fibonacci?

Para números mayores, la paralelización tuvo un impacto positivo, ya que se aprovechan los recursos y las cualidades del algoritmo fibonacci para hacer un trabajo más eficiente.

4. ¿Qué limitaciones encontraste al intentar paralelizar el cálculo?

Intenté paralelizar el cálculo con las estrategias bottom-up y top-down con memoization, pero me di cuenta de que sus ventajas no se aprovechan como en el cálculo secuencial y terminan siendo contraproducentes.

El top-down con memoization funciona mejor con hilos, porque comparten memoria y pueden reutilizar los cálculos ya hechos. Sin embargo, esa mejora viene de la memoization y no del paralelismo, y además puede haber condiciones de carrera (dos hilos calculando y guardando el mismo valor). Con procesos, cada uno tiene su propia memoria, por lo que el arreglo memoization no se comparte y se gastan recursos en crearlo para cada proceso.

El bottom-up tiene una dependencia de datos entre iteraciones (cada valor depende de los dos anteriores), así que su ciclo interno no se puede paralelizar. Lo único que se podría paralelizar es llamarlo para cada i , pero cada llamada reconstruye su propia tabla, y el cálculo es tan corto que el costo de crear hilos o procesos supera el beneficio.

5. ¿Cómo escalaría tu solución con un número mayor de hilos/procesos o un valor más grande de N ?

Con un valor más grande de N , la ventaja de los procesos e hilos frente a la versión secuencial es más evidente, porque el número de operaciones crece exponencialmente y el costo de ejecutarla en paralelo es considerablemente menor a su versión secuencial.